UNIT 2 (Continued)

UNIT 2 (Continued) — ER Model: Mapping, Constraints, and Extended ER

---

1. Mapping of ER Diagram into Relations

Once an ER diagram is designed, it needs to be converted into a set of relational tables (schemas) so that it can be implemented in a database system. This process is called **mapping** or **reduction to relational schemas**. The general rules are as follows:

1.1 Mapping a Strong Entity Set

A strong entity set is directly mapped to a table. Each attribute of the entity becomes a column in the table. The primary key of the entity set becomes the primary key of the table.

Example:
The entity set `Student(Roll_No, Name, Branch, Year)` is mapped as:

```
Student(Roll_No, Name, Branch, Year)
         ↑ Primary Key
```

1.2 Mapping a Weak Entity Set

A weak entity set does not have its own primary key, so it must borrow it from its identifying (owner) entity set. The resulting table contains:

- All attributes of the weak entity set
- The primary key of the identifying entity set (as a foreign key)
- The primary key of the resulting table = PK of identifying entity + discriminator of weak entity

Example:
`Loan` is a strong entity with primary key `Loan_No`.
`Payment` is a weak entity (dependent on Loan) with discriminator `Payment_No`.

Mapping:

```
Loan(Loan_No, Amount, Date)
Payment(Loan_No, Payment_No, Payment_Date, Amount)
              ↑FK from Loan    ↑ Together form PK
```

1.3 Mapping a Binary Relationship Set

The mapping depends on the cardinality ratio:

Many-to-Many (M:N):
A new separate table is created for the relationship. This table contains the primary keys of both participating entity sets (as foreign keys), which together form the primary key of the new table. Any descriptive attributes of the relationship are also included in this table.

Example:
`Student` enrolls in `Course` (M:N relationship with attribute `Grade`):

```
Enrollment(Roll_No, Course_ID, Grade)
             ↑FK        ↑FK
             ← composite PK →
```

One-to-Many (1:N):
No new table is needed. The primary key of the entity on the "one" side is added as a foreign key in the table on the "many" side.

Example:
One `Department` has many `Students`:

```
Student(Roll_No, Name, Dept_ID)
                         ↑ FK referencing Department(Dept_ID)
```

One-to-One (1:1):
No new table is needed. The primary key of one entity is added as a foreign key in the other entity's table. Typically, the foreign key is placed in the table whose entity has total participation in the relationship.

Example:
`Employee` manages `Department` (1:1):

```
Department(Dept_ID, Dept_Name, Manager_Emp_ID)
                                    ↑ FK referencing Employee
```

1.4 Mapping a Multivalued Attribute

A multivalued attribute cannot be stored directly in a single cell. A new separate table is created containing:

- The primary key of the original entity (as a foreign key)
- One column for the multivalued attribute

Example:
`Employee` has multiple `Phone_Numbers`:

```
Employee_Phone(Emp_ID, Phone_No)
               ↑FK     ← both together form PK
```

1.5 Mapping a Composite Attribute

Composite attributes are broken down into their simple (atomic) component attributes. Only the atomic sub-attributes are stored as separate columns in the table. The composite attribute name itself is not stored.

Example:
`Name` (composed of `First_Name`, `Middle_Name`, `Last_Name`) → stored as three separate columns:

```
Employee(Emp_ID, First_Name, Middle_Name, Last_Name, ...)
```

1.6 Mapping a Derived Attribute

Derived attributes (those computed from other stored attributes) are generally **not stored** in the database. They are calculated at query time when needed.

Example:
`Age` is derived from `Date_of_Birth`. Only `Date_of_Birth` is stored; `Age` is computed when required.

---

2. Participation Constraints

Participation constraints describe whether the existence of an entity depends on it being related to another entity via a relationship. There are two types:

2.1 Total Participation

An entity set has **total participation** in a relationship if **every** entity in that entity set must participate in at least one instance of the relationship. In other words, no entity in the set can exist without being associated with some entity in the other set through that relationship.

Total participation is indicated in an ER diagram by a **double line** connecting the entity set to the relationship set.

Example:
Every `Loan` must have a `Borrower`. The entity set `Loan` has total participation in the `Borrower` relationship. If a loan record exists in the database, it must be associated with at least one customer.

2.2 Partial Participation

An entity set has **partial participation** in a relationship if only **some** entities in that entity set participate in the relationship. There can be entities that do not participate in any relationship instance.

Partial participation is indicated by a **single line** connecting the entity set to the relationship set.

Example:
Not every `Customer` is required to have a `Loan`. A customer can exist in the database without having taken any loan. So `Customer` has partial participation in the `Borrower` relationship.

2.3 Expressing Participation Using Cardinality Limits

Participation constraints can also be expressed using **min-max notation** on the lines of an ER diagram. The notation `(min, max)` is written on the line connecting the entity set to the relationship:

- `min = 0` means partial participation (an entity need not participate)
- `min = 1` (or more) means total participation (every entity must participate)
- `max` indicates the maximum number of relationship instances an entity can participate in

Example:
An `Employee` must work in at least one `Department` and at most one: notation `(1,1)` on the employee side. A `Department` may have one or more employees: notation `(1, N)` on the department side.

---

3. Extended ER (EER) Model — Generalization, Specialization, and Aggregation

The basic ER model is sufficient for many applications, but certain real-world scenarios require richer modeling. The **Enhanced (Extended) Entity-Relationship Model** adds three key concepts: Specialization, Generalization, and Aggregation.

---

4. Specialization

Specialization is a **top-down** design process. It involves taking a single, higher-level entity set and breaking it down into more specific, lower-level sub-groupings called **subclasses**, based on some distinguishing characteristic.

The motivation for specialization is that certain entities in an entity set may have attributes or relationships that are not shared by all other entities in the set. Rather than giving those attributes to every entity (which would result in many NULL values), we define subclasses.

How it works:
An entity set (the **superclass**) is identified first. Then, subsets of that entity set that have distinct properties are identified and defined as subclasses. Each subclass:

- Inherits all attributes and relationships of the superclass
- Has its own additional (specific) attributes

Example:
Consider a superclass `PERSON` with attributes `Name`, `Street`, `City`. A person may be further classified as either a `CUSTOMER` or an `EMPLOYEE`. These are subclasses of PERSON. A `CUSTOMER` may additionally

have attributes like `Credit_Rating`, and an `EMPLOYEE` may additionally have attributes like `Salary` and `Employee_ID`. Furthermore, `EMPLOYEE` can itself be specialized into `OFFICER`, `TELLER`, and `SECRETARY`.

- `OFFICER` has an additional attribute `Office_Number`
- `TELLER` has additional attributes `Station_Number` and `Hours_Per_Week`
- `SECRETARY` has an additional attribute `Hours_Per_Week`

Representation in EER Diagram:
Specialization is depicted using a **triangle** labeled **ISA** (meaning "is a"). The superclass sits above the triangle, and the subclasses are connected below it. The ISA label reflects relationships such as: "A CUSTOMER is a PERSON", "An EMPLOYEE is a PERSON".

Higher-level and lower-level entity sets are both represented as rectangles.

Key Properties of Specialization:

- The same superclass can have **multiple specializations** based on different criteria.
  - Example: EMPLOYEE can be specialized into `{SECRETARY, ENGINEER, TECHNICIAN}` based on job type, AND also into `{SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}` based on payment method.
- Attributes that belong only to a subclass are called **specific attributes**.
  - Example: `TypingSpeed` is a specific attribute of SECRETARY.
- A subclass can also participate in **specific relationship types** not applicable to the full superclass.

---

5. Generalization

Generalization is a **bottom-up** design process. It is the exact reverse of specialization. In generalization, multiple entity classes that share common features are combined and abstracted into a single, higher-level entity set called a **superclass**. The original entity classes then become subclasses of this new superclass.

Generalization helps eliminate redundancy by identifying the common attributes and relationships shared across several similar entity sets and grouping them at the superclass level.

Example:
Suppose we have two separate entity sets, `CAR` and `TRUCK`, each with their own attributes. On examination, we find they share common attributes like `Vehicle_ID`, `Speed`, `License_Plate`, and `Fuel_Type`. We generalize them into a superclass called `VEHICLE`. Both `CAR` and `TRUCK` become subclasses of `VEHICLE` and inherit its common attributes.

- CAR still retains its specific attribute: `No_of_Passengers`
- TRUCK still retains its specific attribute: `Tonnage`

Important Note:
Specialization and generalization are simply two perspectives on the same concept. Whether we are going from the general to the specific (specialization) or from the specific to the general (generalization), the resulting EER diagram looks identical. The two terms are often used interchangeably.

---

6. Attribute Inheritance

Attribute inheritance is the mechanism by which subclasses automatically acquire the properties of their superclass.

A lower-level entity set (subclass) **inherits**:

- All the **attributes** of the higher-level entity set (superclass)
- All the **relationship participations** of the superclass

This means we do not need to re-list the superclass attributes for each subclass. The subclass only defines its own **additional (specific) attributes**.

Example:

- `PERSON` has: `Name`, `Age`, `Address`
- `EMPLOYEE` (subclass of PERSON) inherits `Name`, `Age`, `Address` and additionally has `Emp_ID`, `Salary`
- `STUDENT` (subclass of PERSON) inherits `Name`, `Age`, `Address` and additionally has `Roll_No`, `Course`

Types of Inheritance

Single Inheritance: A subclass has exactly one superclass. In a hierarchy, a given entity set is a lower-level entity set in only one ISA relationship.

Multiple Inheritance: A subclass has more than one superclass, meaning the entity set is a lower-level entity set in more than one ISA relationship. The resulting structure is called a **lattice**.

Example of Multiple Inheritance:
`STUDENT_ASSISTANT` is both an `EMPLOYEE` (it earns a salary) and a `STUDENT` (it is enrolled in courses). So `STUDENT_ASSISTANT` inherits attributes from both superclasses.

---

7. Constraints on Specialization and Generalization

Two types of constraints govern how specialization and generalization work.

7.1 Disjointness Constraint

This constraint specifies whether a single entity from the superclass can be a member of **more than one** subclass of the same specialization.

Disjoint (symbol: `d` in the EER diagram):
An entity can be a member of **at most one** subclass. The subclasses are mutually exclusive.

Example: An EMPLOYEE can be classified as either SALARIED or HOURLY, but not both at the same time. This is a disjoint specialization.

Overlapping (symbol: `o` in the EER diagram):
An entity can be a member of **more than one** subclass simultaneously.

Example: A PERSON can be both a STUDENT and an EMPLOYEE at the same time. This is an overlapping specialization.

7.2 Completeness Constraint

This constraint specifies whether every entity in the superclass **must** belong to at least one subclass.

Total Completeness (shown by a double line from superclass to triangle):
Every entity in the superclass must be a member of at least one subclass. No superclass entity can exist outside the subclasses.

Example: Every SHAPE must be either a CIRCLE, RECTANGLE, or TRIANGLE — no shape can exist without belonging to one of these categories.

Partial Completeness (shown by a single line from superclass to triangle):
An entity in the superclass may or may not belong to any subclass. Some entities can exist without belonging to a subclass.

Example: Not every EMPLOYEE is necessarily a MANAGER — some employees don't fit into any defined subclass.

Combining the Two Constraints

The two constraints combine to give four possible scenarios:

Combination	Meaning
Disjoint + Total	Every superclass entity belongs to exactly one subclass
Disjoint + Partial	A superclass entity belongs to at most one subclass (possibly none)
Overlapping + Total	Every superclass entity belongs to at least one subclass (may belong to more)
Overlapping + Partial	A superclass entity may belong to zero, one, or more subclasses

8. Aggregation

Aggregation is an abstraction concept used in the EER model to handle situations where a **relationship itself needs to participate in another relationship**. In standard ER modeling, relationships cannot directly participate in other relationships. Aggregation solves this limitation.

In aggregation, a relationship set (along with the entities it involves) is treated as a single, higher-level abstract entity. This aggregated unit can then participate in other relationships like any normal entity.

Why is it needed?
Consider the following scenario: an `EMPLOYEE` works on a `PROJECT`. This is a relationship called `WORKS_ON`. Now, suppose each instance of an employee working on a project also involves the use of a specific `MACHINE`. The `USES` relationship is between the combination of (Employee + Project working-together) and Machine — not just between Employee and Machine, or Project and Machine independently.

In standard ER, we cannot directly link a relationship to another entity. Aggregation allows us to treat the `WORKS_ON` relationship (and its participating entities EMPLOYEE and PROJECT) as a single abstract entity. This aggregated entity can then participate in the `USES` relationship with MACHINE.

Representation in EER Diagram:
The relationship set to be aggregated, along with its participating entity sets, is enclosed in a **dashed rectangle**. This dashed rectangle is then treated as an entity box that can be connected to other relationships.
